

Seeds: Protocol Specification

Birdbrain

<https://birdbrain.wtf>

Technical specification · 6 October 2026 · Version 3.3 · runtime seeds, spec 102

Abstract. Seeds is a chain that records who joined a community, what they put forward and which of it stood. A member is admitted when enough existing members witness them against the same session evidence. A contribution is a digest that matures unless enough members challenge it. Units are minted when a contribution matures and nowhere else: joining mints nothing, there is no bond and units do not move between members. The runtime is five pallets: four from a standard Substrate solo chain and one application pallet of about a thousand lines. It has no balances pallet, no fees, no sudo and no treasury. This paper specifies that pallet as definitions, invariants and algorithms. It reports a run of twenty-eight checks on a three-node network, derives from the constants what a colluding set of members can do, designs private ballots, challenges and contributions that keep a member's acts unlinked until they choose to link them, and lists what is not built. The argument for the design is in the companion paper [1].

1 Scope

Identity online is either issued by an institution or held as a key, and neither records what a person did or who will stand behind them. Seeds replaces issuance with witnessed admission and records contributions against the members who made them. The companion paper makes that case in plain terms [1]. This one specifies the runtime.

1.1 Why one pallet

A general-purpose chain accumulates modules. Accounts need fees, so a balances pallet appears. Fees need a destination, so a treasury appears. The treasury needs spending, so bounties and councils appear. Upgrades need an administrator, so a sudo key appears. Each step is reasonable. Together they mean that issuance is no longer one public function, and the chain will record whatever can be put through a pallet. Generality is the right objective for a platform that hosts every application. It is the wrong one for a ledger whose whole job is to say who joined and what stood.

Seeds therefore has one application pallet and refuses the rest.

There is no `pallet_balances`, no `pallet_transaction_payment`, no `pallet_sudo`, no treasury, no bounties and no contracts. Account data in `frame_system` is empty, and balances of the unit live in Seeds. The pallet depends only on `frame_system` and a validator-set trait, so moving it to another FRAME-compatible base is a rename. The source is public [5].

Table 1: Runtime composition. Everything but Seeds is the Polkadot SDK solo-chain template [2], less what fees and balances needed.

Pallet	Job
<code>frame_system</code>	blocks, extrinsics, account nonces, authorised upgrades
<code>pallet_timestamp</code>	wall clock for slots
<code>pallet_aura</code>	block production [3]
<code>pallet_grandpa</code>	finality [4]
<code>pallet_seeds</code>	admit, record, mint, govern, seat

2 Model

2.1 Three relations

Three questions look similar and are not.

Definition 2.1 (Control). A party *controls* an account id if they can produce a proof π that the chain accepts for a message m under id. Control is a cryptographic fact.

Definition 2.2 (Membership). An account is a *member*, written $\text{id} \in \text{Mem}$, if it was named at genesis or admitted under the rule of Section 4. Membership is a social fact that the chain records and cannot originate.

Definition 2.3 (Holding). An account *holds* k units if the chain’s balance map says so. Holding is an arithmetic fact, produced only by the mint and claim rules of Section 6. Units do not move between accounts.

Invariant 2.1 (Separation). Control of id does not imply $\text{id} \in \text{Mem}$. Membership of id does not imply a positive balance. A positive balance does not imply the right to vote, to witness or to hold a seat.

Invariant 2.2 (The right to write is membership). A signed extrinsic whose signer is not in Mem is invalid before it reaches the pool. Unsigned extrinsics are a closed list with one entry, claim.

Invariant 2.2 is enforced by a transaction extension, `OnlyMembers`, placed before the nonce check. It reads the members map and returns `BadSigner` if the signer is absent. Unsigned extrinsics pass to `ValidateUnsigned`, which accepts only a well-formed claim. Membership replaces fees as the guard against spam. If the unit is a participation record, charging it as postage would turn the record into fuel.

2.2 Participants, time and communities

The system has members, candidates and holders of snapshot claims. A candidate is an account with an open candidacy. A holder of a snapshot claim is an old-chain key with an entry in the claims map, and need not be a member. Genesis names a opening set, who are members of community 0.

Time is block numbers, with a block every six seconds. An *era* is a run of L blocks, $\text{era}(n) = \lfloor n/L \rfloor$. Allowances, admission caps and seating turn over on the era.

A community is an admission queue with a cap per era. Each member belongs to the one community they were admitted into. Further communities are opened by motion. Once a community has at least w members, where w is the witness threshold, witnesses for it must come from inside it. Until then any member may witness, so that a new community can start at all. A community has no account and no motions of its own.

2.3 What the chain does not decide

Uniqueness. Two keys can belong to one body, and the chain has no body to inspect. Systems that try to decide uniqueness (biometrics, video, ink, registries of unique humans) sit below Seeds, as a floor it could require [6, 7]. Seeds does not implement that floor. It implements what sits above: standing that comes from continuing, witnessed participation, and a contribution that traces to the member who made it. Aliases that cannot be linked across contexts protect privacy well and carry attribution badly, because a contribution cannot credit or challenge an alias that cannot be joined to the next context. Seeds is private by default and linkable by consent: an act carries only the identity its rule needs, and the member decides when to join their acts together (Section 12).

Content. A contribution’s content is not on the chain. The chain holds a 32-byte digest, a clock and a set of challengers.

Money. Dollars are not in the model. Section 11 says why a dollar on this runtime would change its security.

3 Ownership

An account is a 32-byte identifier. The chain does not care which kind of key stands behind it. It cares whether a proof shows control of that identifier for one message. Each kind of key is an *ownership type*, a variant of an enum, so adding a scheme adds a variant and moves nobody’s balance. The idea is borrowed from Locus [8]. Ownership types say who controls a balance. They say nothing about who is a member.

Definition 3.1 (Ownership proof). An ownership proof π for identifier id and message m is one of

$$\pi \in \{\text{Sr25519}(\sigma), \text{Ed25519}(\sigma), \text{Ecdsa}(\sigma), \text{Passkey}(\alpha)\}.$$

$\text{proves}(\pi, \text{id}, m)$ holds only if the recovered or supplied public key maps to id and the signature is valid over m , or over the browser-wallet wrapping $\langle \text{Bytes} \rangle m \langle / \text{Bytes} \rangle$.

Table 2: Ownership types. Passkey identifiers are domain-separated so a P-256 key cannot collide with an identifier of another type.

Type	Identifier	Signed over
Sr25519, Ed25519	the public key	m , raw or wrapped
ECDSA (secp256k1)	blake2_256 of the compressed public key	blake2_256(m), raw or wrapped
Passkey (P-256, WebAuthn)	passkey_id of the compressed public key	an assertion whose challenge is sha256(m)

3.1 Passkeys

With the passkey type the private key never leaves the device’s authenticator, and the chain checks the authenticator’s own signature.

Definition 3.2 (Passkey identifier). Let P be a 33-byte SEC1 compressed P-256 public key. Then

$$\text{passkey_id}(P) = \text{blake2_256}(\text{seeds/passkey-p256:} \parallel P).$$

An assertion α carries P , the authenticator data, `clientDataJSON` and a 64-byte signature $r \parallel s$. Verification requires all of the following.

1. The authenticator data is at least 37 bytes and the user-present flag is set.
2. `clientDataJSON` begins with the WebAuthn Level 3 limited-verification prefix

$$\{"type": "webauthn.get", "challenge": " \parallel \text{b64url}(\text{sha256}(m)) \parallel "$$

which binds the assertion to m [9].

3. The P-256 signature verifies over $\text{sha256}(\text{authenticator data} \parallel \text{sha256}(\text{clientDataJSON}))$, with s normalised.
4. $\text{passkey_id}(P) = \text{id}$.

The check binds the challenge and user presence. It does not re-implement a browser’s origin policy. P-256 verification runs in Wasm with no host function. It measured about 1.6 ms on a machine at 106–108% of the FRAME reference CPU and is priced at 2.5 ms until a benchmark replaces the constant.

3.2 The claim

A network may start from balances snapshotted on an earlier ledger. Those balances move only by claim. The one unsigned call is `claim(dest, old,  $\pi$ )`, over the message

$$m = \text{seeds-claim:} \parallel h_0 \parallel \text{SCALE}(dest),$$

where h_0 is the hash of the network's genesis block. Two networks may carry the same snapshot. Binding the message to h_0 stops a claim signed for one being replayed on the other. Any ownership type that controls *old* will do. The validator checks that *old* still has a claim and that π verifies, and tags the transaction with *old* so it cannot be replayed. Genesis places the snapshot in the claims map and in total issuance, and tracks the unclaimed remainder. A claim moves units from unclaimed to *dest*. It does not mint, and *dest* need not be a member. A migration approved only by operators would bypass the holders. A claim the old key must sign cannot.

4 Admission

Admission is the only path into Mem after genesis.

4.1 Evidence and candidacy

A witness names a candidate, a community and a 32-byte evidence hash. The hash is meant to commit to a session the witnesses attended, such as the root of its attendance record. The chain does not interpret it. It enforces that enough distinct members name the same hash for the same candidate in the same community, inside a timeout.

The sessions held so far already produce such a root. Each attendee signs in with a passkey, and the sign-in log is built into a Merkle tree whose leaves bind a person, an account and the session. An inclusion proof then shows that an account attended. The pallet does not ask for one. A witness today therefore asserts presence as well as personhood, and only the second needs a human (Section 16).

The rule is written to follow a meeting a community already holds and to add no step to it. Signing in is how a person enters the session, so presence costs an attendee nothing and is computed from the log. In the runtime specified here **witness** is still an ordinary signed call that names the hash, so admission needs two members to act. Section 4.2 sets out how that act can be removed.

4.2 Computed admission (proposed)

The attendance record is a Merkle root built from the session's sign-in log. Each leaf binds $\text{blake2}(\text{person})$, the account and $\text{blake2}(\text{session})$, and a root naming one person with two accounts, or one account for two people, is refused when built. The root is fixed before anything is counted against it, and the roster behind it is published, so anyone can rebuild the root and dispute it. Continuity is the rule that turns these records into admission: a person counts after two attendances, the second witnessed by members present at the first.

This separates two standings. Participation is attendance: a key in a matured root is a participant, recorded with no vote and no write access beyond what the profile allows. Membership is derived from consistency across roots, by the continuity rule below. Carried

into the pallet, that becomes three changes.

1. **Records as points.** A member who attended posts the session's root with `propose_point`. It matures by the ordinary rule, and θ members who appear in the root as attendees can burn it.
2. **Admission by continuity.** A call carrying inclusion proofs admits candidate c when two matured roots r_1, r_2 for the same community both contain c and both contain at least k common members. Those k members are recorded as c 's witnesses, for strikes and allowances, exactly as if they had called `witness`. Any account may submit the call, since the proofs carry the authority.
3. **Challenge by exception.** The judgement a witness makes today, that a key is a person and not a second key for someone counted, is exercised by challenging the root, or a later point naming a leaf, inside its window.

None of this is built. Two costs come with it. The per-member allowance and strikes now attach to members who were merely present, which is fair only if they were able to check the record. And the roster that maps a sign-in to a person is still compiled by whoever builds the root, so it is an issuer by another name until the root itself can be challenged, which is the first change.

Definition 4.1 (Candidacy). A candidacy for account c is a tuple (q, e, W, t_0) : a community, an evidence hash, a bounded set of witnesses and the block it opened. While it is open, a witness naming a different q or e is refused.

4.3 Four brakes

1. **Threshold.** w distinct members must witness. Lab value $w = 2$.
2. **Allowance.** A member may witness at most a candidates per era. Lab value $a = 3$.
3. **Cap.** A community admits at most $cap(q)$ people per era. Genesis sets the opening cap at 10, and motions change it.
4. **Strikes.** When a member is expelled, each of their witnesses takes a strike, and a member whose point fails takes one too (Section 5). At $s_{\max}$ strikes a member may no longer witness or propose. Lab value $s_{\max} = 2$.

Section 11 shows which of these binds against a colluding set. It is the cap.

4.4 What admission does

Admission assigns a network-wide index, records the community, the evidence, the witnesses and the block, and gives the account a provider reference in `frame_system`, since no balances pallet exists to do so. It mints nothing. First members are admitted at genesis with empty evidence and the first-member flag set. The index is the seating order of Section 8.

Algorithm 4.1: Witness and possibly admit

Input: signer b , candidate c , community q , evidence e

```

1 if  $b \notin \text{Mem}$  or  $c \in \text{Mem}$  then refuse;
2 if  $\text{strikes}(b) \geq s_{\max}$  then refuse;
3 if  $q$  unknown then refuse;
4 if  $|q| \geq w$  and  $\text{community}(b) \neq q$  then refuse // outside witness;
5 if allowance of  $b$  for era(now) is spent then refuse;
6 open or resume the candidacy of  $c$  for  $(q, e)$  inside the timeout;
7 if  $b \in W$  then refuse;
8  $W \leftarrow W \cup \{b\}$  and consume one allowance of  $b$ ;
9 if  $|W| \geq w$  then
10   if  $q$  is at its era cap then refuse;
11   increment the era counter of  $q$ ;
12   admit( $c, q, e, W$ );

```

4.5 Expulsion

Expulsion is a motion and never an administrative act. If it passes, the member is removed, their session keys are dropped, any open motion of theirs is cancelled, each of their witnesses takes a strike and the community count falls. Their units are not seized.

5 Points

A contribution reaches the chain as a *point*.

Definition 5.1 (Point). A point with digest d is a tuple $(p, t_{\text{prop}}, t_{\text{mat}}, C, \nu, \sigma)$, where p is the proposing member, C a bounded set of challengers, ν an optional note of at most N bytes, and $\sigma \in \{\text{Pending}, \text{Matured}, \text{Failed}\}$.

1. A member proposes d . There is no bond and no fee. The point stays pending until $t_{\text{mat}} = t_{\text{prop}} + T$. A member may have at most Q points pending, and at most R points may mature in one block.
2. Any other member may challenge once. A proposer cannot challenge their own point.
3. If $|C|$ reaches the threshold θ before then, the point fails. Nothing is minted, and the proposer takes a strike.
4. At t_{mat} , a pending point whose proposer is still a member matures and M is minted to the proposer. If the proposer has been expelled, it fails.

A failed point costs standing rather than units, by the same strike that stands down a witness whose admittee is expelled (Section 4). A member with $s_{\max}$ strikes may neither witness nor propose. Lab values are $Q = 3$, $T = 5$ minutes, $\theta = 3$, $R = 16$ and $N = 128$.

Maturity is an optimistic confirmation: nobody who could object did, in number, in time. It does not show that the content is true, useful or original. The digest is what keeps the

Algorithm 5.1: Mature a due digest, called from `on_initialize`

Input: digest d

- 1 let P be the point at d ;
 - 2 **if** P is missing or not pending **then** return;
 - 3 **if** the proposer of P is a member **then**
 - 4 | mark matured, release one pending slot of p , $\text{mint}(p, M, \text{Maturity})$;
 - 5 **else**
 - 6 | mark failed;
-

chain small. A graph, a transcript or a dataset each reduces to 32 bytes, and the chain's work is the clock and the count of challengers.

6 Issuance

Let M be the maturity mint. The lab value is $M = 10$ units, with one unit equal to 10^{12} base units. The runtime calls the unit an amount and nothing else, and each network names its own.

Definition 6.1 (Mint). $\text{mint}(\text{id}, k)$ adds k to the balance of id and to total issuance. It has one call site, `mature`. Admission mints nothing, there is no bond to debit or burn, and there is no transfer.

Invariant 6.1 (Issuance is a public function). Let I_t be total issuance after block t , and $m_t \leq R$ the points that matured in it. Then

$$I_t - I_{t-1} = m_t M.$$

No other term exists.

Invariant 6.2 (Issuance is at the edge). Every unit added under [Invariant 6.1](#) is credited to the proposer of a newly matured point. None is credited to a treasury, a fee sink, an administrator, a block producer or a newly admitted member. Consensus is not paid in units, and votes are not counted in units.

Summing [Invariant 6.1](#) from genesis gives a conservation identity. With m matured points and snapshot total S ,

$$I = m M + S. \tag{1}$$

Claims move units out of the unclaimed part of S without changing I . Issuance never falls. Anyone who can read the chain can recompute [Eq. \(1\)](#), and if it fails the pallet is wrong.

6.1 What changed from the first runtime

The first runtime also minted $A = 100$ units on admission, made a proposer post a bond of at least one unit that was burned if the point failed, and let a member transfer units to

another member. All three are gone. A member holds what they have added, and the unit counts continued participation and not arrival. None of the earlier specifications this design descends from minted for joining. Three consequences follow.

1. **The cost of a bad point** is standing, not units. A member admitted with an empty balance can propose at once, and a member who keeps proposing points the room rejects is stood down after $s_{\max}$ failures.
2. **The attacker's prize.** A manufactured member carries away no units. What remains to be won is a vote and a place in the seating order, which is what [Section 11](#) analyses.
3. **The unit does not trade on the network.** With no transfer it is a record of what a member added, not something that changes hands.

Two properties found in earlier specifications are absent from the pallet. Several set aside a tenth of each mint for the members who connect one contribution to another, and some sized each mint to the work by vote. Here every matured point mints the same M , and a point carries no reference to any other, so there is nothing yet for a connecting share to follow. Both are listed in [Section 16](#).

7 Governance

A member may have one open motion at a time. Opening a motion counts the proposer's aye and freezes the electorate at the members whose admission index is below the next index. A vote may be changed until the motion closes.

Definition 7.1 (Motion kind). A motion is one of

$$\begin{aligned} &\text{Upgrade}\{code_hash\}, \quad \text{OpenCommunity}\{cap\}, \quad \text{SetCap}\{q, cap\}, \\ &\quad \text{Expel}\{id\}, \quad \text{EndOpening}. \end{aligned}$$

Let E be the frozen electorate, Y the ayes and N the nays, with

$$\text{vetoed} = (3N \geq \max(E, 1)), \quad \text{majority} = (2Y > E).$$

- **EndOpening** passes on a majority while opening control is active. It is irreversible.
- **Expel** passes on a majority that is not vetoed, if the target is still a member. The first-member rule never applies to it.
- Any other kind, proposed by a first member while opening control holds, passes unless vetoed.
- Otherwise a motion passes on a majority that is not vetoed.

An upgrade motion authorises a code hash, after which any member may submit the matching Wasm. Opening control lets a small set ship the first upgrades before a majority

exists. Members can end it, and the pallet will not restart it.

Proposition 7.1 (No silent administrator). *Every change to runtime code, community caps, membership or the opening flag is either a genesis fact or the enactment of a closed motion. The pallet has no other origin.*

8 Seating

Block production is Aura, one authority per six-second slot, and finality is GRANDPA. The pallet implements neither. It produces a list of session keys and hands it to a runtime trait.

Definition 8.1 (Seating). Let H be the members who have registered session keys, ordered by admission index, oldest first. The seated set K is the first $\min(|H|, V_{\max})$ of them. Lab value $V_{\max} = 21$.

On each era boundary the pallet builds K . If K is empty or unchanged, nothing happens. Otherwise the runtime applies it to GRANDPA first, and to Aura only if GRANDPA accepted, so the two sets never disagree. GRANDPA refuses while a change is pending, and the pallet retries at the next era.

Registering keys is a member call. It needs no vote, no stake and no payment. Seating is not random and not elected. The scarce resource is the admission index, so a member admitted late sits behind everyone admitted earlier who holds keys.

$V_{\max}$ is a parameter. Some bound is needed because the cost of a GRANDPA round grows with the square of the set and an absent machine in a small set stalls finality. The consensus pallets accept up to 32 authorities. The value 21 is a choice.

9 Constants

All extrinsic weights are set by hand. None has been benchmarked.

10 Results

The runtime was started from genesis on three nodes on one machine, with three first members, two of them seated, and one snapshot claim of 1,000 units. A script then ran the twenty-eight checks of Table 4 between blocks 1 and 120, and all passed unattended.

Check 25 is Eq. (1) with $m = 1$ and $S = 1000$, giving 1,010 units. The pallet also has 31 unit tests. This is a conservation check that anyone can repeat. It is not a security proof.

11 Security

11.1 What an attacker wants

1. To mint units without a maturity.

Table 3: Lab constants. They are short so that a full cycle fits one sitting. A network that mattered would take an era of about a day and a maturity period of weeks.

Constant	Lab value	Role
Slot	6 s	block production
Era length L	10 minutes	allowances, caps, seating
Witnesses required w	2	admission threshold
Witness allowance a	3 per era	brake on each member
Candidacy timeout	1 hour	stale candidacies
Maximum strikes $s_{\max}$	2	member stands down
Opening cap	10 per era	community 0
Maturity mint M	10 units	the only mint
Pending points per member Q	3	brake on each proposer
Maturity period T	5 minutes	challenge window
Challenge threshold θ	3	a point fails
Maximum maturing per block R	16	weight bound
Maximum note N	128 bytes	on-chain remark
Voting period	2 minutes	motions
Maximum seats $V_{\max}$	21	seated set
Maximum key holders	1,000	queue bound

2. To take a seat without being a member in admission order.
3. To pass a motion without the electorate.
4. To move a snapshot balance without the old key.
5. To manufacture members cheaply enough that seats or motions follow.

The first four are closed in the pallet. Mint has one call site. Seating reads the key-holder list in index order. Motions freeze the electorate when they open. A claim needs a verifying ownership proof and consumes its entry. The fifth is the real question.

11.2 What a colluding set can do

The chain cannot test that an evidence hash commits to a session that took place. A set of members acting together can witness a fiction. Computed admission (Section 4.2) would narrow this to sessions whose record has itself stood unchallenged. The brakes are then the whole defence, and one of them does not bind.

Proposition 11.1 (The allowance does not bind a colluding set). *If $a \geq w$ and a new member may witness in the era of their admission, then w colluding members in one community can admit candidates until the community’s cap for the era is reached.*

Each admission spends w allowances from the set and adds a member who brings a fresh ones, so the stock of allowances never falls. With $w = 2$ and $a = 3$ it grows by one with every admission. The allowance still limits what one honest member can be talked into, and it

Table 4: The end-to-end run of spec 102, 5 October 2026.

#	Check
1	The runtime is five pallets and nothing else
2–6	Genesis: three first members holding nothing, no transfer call, opening control on, two seats
7	A non-member’s signed transaction is refused before the pool
8	One witness does not admit
9	A second witness naming different evidence is refused
10–11	A second witness naming the same evidence admits the candidate, and nothing is minted
12	The new member writes without a fee
13	A claim signed for another network’s genesis is refused
14	A claim signed for a different destination is refused
15–16	The old key moves 1,000 units to the new member, who signs nothing, and nothing is left unclaimed
17	The new member registers keys and joins the queue
18	A point is proposed with no bond, and one challenge leaves it pending
19	Three challenges fail another point: nothing is minted and its proposer takes a strike
20	A first member’s upgrade motion, unopposed, is authorised
21	A member applies the authorised code, spec 102 to 103, with no sudo and no restart
22	Members end opening control, three of four
23–24	The first point matures and mints 10 units, the only mint of the run
25	Total issuance is exactly $10 + 1000$ units
26–27	At the era boundary the new member’s keys join Aura, and Seeds records three seats
28	GRANDPA finalises past the set change with three voters (block 117)

makes each false member name its witnesses for the strike rule. The rate of a colluding set is governed by the cap alone.

Table 5 takes two colluding members, a cap of ten per era and h honest members in one community. After t eras the set controls $2 + 10t$ votes. It can veto, and so block its own expulsion, once it holds a third of the electorate, which is when its size reaches $h/2$. It has a majority once its size exceeds h .

Table 5: Eras a colluding pair needs, at a cap of ten admissions per era, if no honest member acts. With an era of a day, read the columns as days.

Honest members h	Eras to a veto	Eras to a majority
10	1	1
20	1	2
50	3	5
100	5	10
500	25	50
1,000	50	100

Three things follow. The time to capture grows in proportion to the honest membership, so a small network is the exposed one. The attack is public while it runs: every admission names its witnesses and its evidence, and it uses up the cap that real joiners need. And the window for expelling the set closes when it reaches a veto, so the honest members have to act within the first column.

The cap is per community, and motions can open communities and raise caps, so the

network-wide rate is the sum of the caps. During the opening period a first member's motion to do either passes unless a third object, which makes that period the weakest.

Two changes would strengthen this. A waiting period before a new member may witness would make the allowance bind, limiting a set to growth by a factor of $1 + a/w$ per era. A floor beneath Seeds that made a witness costly in bodies and not only in keys would raise the price of each false member. Neither is built.

11.3 Seats

Seating by admission order protects seats better than votes. GRANDPA finalises with more than two thirds of the seated set. Once fifteen honest members hold keys, later arrivals can take at most six of twenty-one seats, which is too few to stall finality. Once twenty-one do, later arrivals take none. Before that point seats are open to whoever registers keys, and a third of a small set can stall the chain.

The same rule has a cost. A twenty-second member with a machine never takes a seat while the first twenty-one keep theirs. Rotation among key holders is not designed.

One person may run several machines under several memberships, so the number of independent seats has to be counted in people. An experiment to measure that on a network holding no value is designed and has not been run.

Liveness has a known failure. On an earlier test network, with two of three seated machines absent, finality stopped. In Aura a dishonest seat can withhold or equivocate in its own slots, which slows the chain and finalises nothing.

11.4 Flooding

Members write without a fee, and weights are set by hand. A member may have at most Q points pending, but other calls are not limited, so a member can still fill blocks to the weight limit with votes and remarks. The response today is an expulsion motion, which takes a voting period. A general rate limit for each member is not built.

11.5 Money

With the unit alone, capturing the chain costs real people's standing and wins a participation record. With a regulated dollar on the same runtime, the same capture would pay in cash, and twenty-one member machines are a weaker guarantee than a relay chain. If a dollar ever arrives it should arrive last, in an optional module that cannot reach membership or the mint, capped by motion.

12 Privacy

The rules so far attribute every act to an account. Attribution is what the protocol is for, but not every act needs it. A contribution needs a holder, because units and strikes land on it. A ballot needs only to be cast once by a member. A challenge needs only to come from three different members. The principle is *private by default, linkable by consent*: an act carries no more identity than its rule needs, and a member can join their acts together later, by their own proof, when they choose.

Invariant 12.1 (Counted, not attributed). A private act changes a tally, a challenge count or a pseudonym’s record exactly as the public act would. It never changes who can be named as having done it. Supply, tallies and challenge counts remain exact and checkable by anyone.

12.1 What is public, and why

The chain holds no names. An account is a key, and a display name is the member’s own choice, made off the chain, with none as the default. Four things stay public because a rule depends on them. The member set, because **OnlyMembers** and expulsion act on accounts. Admission and its witnesses, because the strike rule has to find the witnesses of an expelled member. Seats and session keys, because block production is by named authorities. Motions and their outcomes, because a rule change must be visible to everyone it binds. The machines that keep the record are visible to their peers at the network level, so a member who needs to hide where they are should not run one. The members of an attendance record are committed by a root. The list behind the root is published with names removed, so a key can be shown present without its holder being named.

12.2 One secret, three uses

A member holds a secret s that never leaves their device. It cannot be the passkey’s own key, because that key stays inside the authenticator and cannot be used in a proof. It is derived from the passkey through the WebAuthn PRF extension, which returns the same 32 bytes for a fixed salt after the member’s face, fingerprint or PIN, so nothing is stored. On admission the member registers an identity commitment with their account,

$$c = H(\text{seeds/id} \parallel s).$$

The *member root* R is a Merkle root over the commitments of current members. Expulsion removes the commitment, and the root is rebuilt without it.

Definition 12.1 (Membership proof). For a scope x , a membership proof π_x shows, without revealing s , c or the path, that the prover knows s such that $H(\text{seeds/id} \parallel s)$ is a leaf under R . Its public output is the nullifier

$$n_x = H(\text{seeds/null} \parallel s \parallel x).$$

One member has one nullifier per scope, and the nullifier cannot be linked to c , to the account, or to the same member's nullifier in another scope. The construction is the one Semaphore uses for anonymous signalling [10]. The chain keeps the spent nullifiers of each open scope and refuses a second act under the same one.

Ballots. The scope is the motion, and the root is the one frozen when the motion opened, which is the electorate freeze of Section 7 restated as a root. A ballot carries π_x and aye or nay. Recasting under the same nullifier replaces the earlier ballot, so a vote can still be changed until the motion closes. The tally is public and the voters are not.

Challenges. The scope is the point. A point falls when three distinct nullifiers have challenged it. Each member can still challenge once, and nobody learns which three did.

Pseudonymous contributions. A member may put a point forward under a pseudonym for a context κ , such as one community or one body of work,

$$p_\kappa = H(\text{seeds/pseud} \parallel s \parallel \kappa),$$

with a membership proof that binds p_κ to the same s . A member has exactly one pseudonym per context, so a strike in a context cannot be shed by starting again in it. Points, units and strikes in κ accrue to p_κ . The pending limit Q becomes a rate limit on nullifiers $H(\text{seeds/rate} \parallel s \parallel \kappa \parallel \text{era}(n) \parallel j)$ for $j < Q$, the construction of the rate-limiting nullifier [11].

Definition 12.2 (Link). A link proof shows that a pseudonym p_κ and an account's commitment c come from the same s . Only the holder of s can produce it. Publishing it joins the pseudonym's record to the account.

Linking is how a member claims a body of work, counts a pseudonym's units with their account's, or shows that an act done for them, for instance by an agent acting on their behalf, traces back to them. Linking moves no units. The trail from an act to the member behind it still exists for every private act. It ends at *a member in good standing* and goes further only when the member links.

12.3 What privacy costs

- **Small rooms give little cover.** A private act hides among the members who could have done it. In a community of ten, a ballot of nine to one says a great deal. Cover grows with membership and with acts gathered over time, not with the strength of the proof.
- **Strikes stay in their context.** A strike on p_κ does not follow its holder to another context unless they link. The two-strike rule binds within a context and is weaker across them. A community that wants it whole can require contributions under accounts and leave ballots and challenges private.
- **Expulsion needs an account.** A motion can expel an account, not a pseudonym. A pseudonym can be barred from its context, which spends its rate nullifiers, but whoever holds it remains a member.

- **Private calls are unsigned.** A signature would name the signer, so private acts join claim as unsigned extrinsics, admitted only with a proof that verifies against the current root and an unspent nullifier. Proof verification costs more than a signature check, and no weight has been measured for it.

12.4 Status

None of this is in the reference runtime. The membership proof with a nullifier has been built as a circuit in Noir and run outside the chain, over the committed attendance of one real session, including attacks that must fail. Verifying such proofs natively has been shown on PolkaVM [12], which is the route on a JAM base (Section 13). On the Substrate runtime it needs a verifier in the pallet or a host function. Display names off the chain, with none as the default, are already how the test network shows its validators.

13 Where it runs

The protocol is a set of rules, and a chain is one machine that can run them. Three placements are possible, and they can run at the same time.

As its own network. The reference runtime is a standalone Substrate chain of five pallets (Section 1). Its security is the seated set of member machines and nothing else. This is the only placement in which every rule in this paper holds as written, and the only one built.

As services on JAM. JAM runs applications as services rather than as whole chains. The rules map onto its parts without forcing:

Rule	Reference runtime	JAM
Only members write	a transaction extension before the pool	an authorizer
Admission	witness , then caps	signatures checked in refine, caps applied in accumulate
Keeping the record	oldest key holders, up to $V_{\max}$	a designated validator set named by a privileged service

One rule runs this way today. A membership register, compiled as a JAM service, records members on a reduced JAM client of our own, which has run with twenty-one validators [13]. All its validators run in one process on one machine. Admission by witness, points, units, motions and seating are not yet services. Kusama voted in 2025 for an independent, lightweight JAM of its own and named proof of personhood as a way to cut its cost [14]. A Seeds network running as services is one candidate for that.

Anchored to another chain. A network may post its finalised checkpoints, or the roots of its attendance records, to an outside chain so that a stranger can check them against

a ledger the network does not control. An anchor adds a witness. It never adds authority: nothing posted outside can mint, admit, vote or seat.

One constraint on every base. Seats and votes come from membership, never from holding a coin, whether the network's own unit or another network's. Where a shared base seats its own validators by stake, as a full JAM network does, that base provides the ordering and finality and the seating rule of [Section 8](#) does not apply to it. The protocol then asks only that its unit is never accepted as stake there, so that holding it cannot buy a place in line by another route.

14 Protocol and profiles

The protocol is the set of rules this paper specifies. A *profile* is a network that runs them with its own settings, and may add arrangements of its own above them. Keeping the two apart keeps the protocol small. The protocol is the root system and profiles are what grows from it. Whatever a profile adds reads two facts from the protocol, who is a member and which points matured, and nothing a profile adds can write either of them.

Fixed by the protocol. Only members write, except for a claim. New units appear only at contributions that stand, and nowhere else. Admission is by witness against session evidence. Motions on membership and on the protocol's own rules are one member, one vote. No key can bypass a vote. Machines that keep the record belong to members and are seated by standing, never by holding.

Set by a profile. The unit's name, the amount created per contribution, the split between a contribution and those it builds on, the number of witnesses, the allowances and caps, the length of a period and of a challenge window, and the number of seats. [Table 3](#) lists the values the lab uses. A profile may also start from a list of balances claimable from an earlier ledger.

Added above by a profile. Funding, tasks and rewards for completing them, a group's shared map of what it knows, licences on a group's work, prices for querying its record, and payment in money. These are contracts between people, kept off the chain.

Never added by a profile. Anything that mints, gives a vote, or gives a place in line to whoever funds or contracts with a group. A profile that needs one of those is a different protocol, and should say so.

15 Related work

Bitcoin. The debt is the discipline. Bitcoin issues one unit for work and records little else [15]. Seeds issues for a contribution that stood and records little else. Bitcoin's unit is measure, store and reward together. Seeds separates them: its unit is not the security bond and not the franchise. A later module or borrowed asset that was allowed to back the seated set would undo that separation [16].

Proof of personhood. BrightID [17], Proof of Humanity [18], Worldcoin [6] and Proof of Ink [7] each try to decide, once, that an account is a unique human. Polkadot’s **pallet-people** stores such decisions and issues an alias for each context. A decision made once costs an attacker once. Standing that depends on continued witnessing has to keep being paid for. Seeds is offered as a mechanism such a registry could consume, and could require one beneath itself. Soulbound tokens [19] share the idea of social facts that cannot be transferred, and usually still merge key, person and holding in one token.

Merit and decay. Škvorc argues that on-chain governance drifts towards plutocracy, and that merit should be earned in each category and should fade with absence [20]. Seeds already removes the balance from the vote. It does not weight votes by merit. MeritRank [21] and PageRank [22] are the natural machinery if it ever does.

Anonymous signalling. Semaphore proves membership of a group and emits a nullifier per scope, so a member can signal once without being identified [10]. The rate-limiting nullifier extends it to a fixed number of acts per period [11]. Seeds uses both for ballots, challenges and pseudonymous points, and differs in where the group comes from: its member root is grown by witnessed admission, not by an operator adding commitments.

Typed ownership. Locus separates the type of key that controls an asset from the asset [8]. Seeds uses that split for claims.

16 Open questions

1. **Transfer.** Whether units should move between members at all (Section 6).
2. **Delegation.** A member can let an AI agent (software that carries out a task on its own once instructed) sign with their key, and every act is then the member’s. A mandate type, by which a member authorises another key to act for them within a stated scope, is not designed. Without it the chain cannot tell a member from an agent holding the member’s key.
3. **Computed admission.** Admission from matured attendance roots by continuity, with the room’s judgement used by challenge, as set out in Section 4.2. Not built. Open within it: the overlap k , whether presence alone should carry a strike, and how a member is shown a record in time to challenge it.
4. **A waiting period for witnesses**, so that the allowance binds (Section 11).
5. **A rate limit for each member**, against flooding.
6. **Rotation of seats** among key holders. On JAM the designated validator set is the other route (Section 13).
7. **Private acts.** Membership proofs, ballots, challenges and pseudonymous points as set out in Section 12. Open within it: the proof system and its verifier on each base, how often the member root is rebuilt, the smallest community in which private ballots should be allowed, and whether a strike should follow a member across contexts without a link.

8. **Registration on install.** The pallet seats whoever has registered keys. Making installation of the software the registration would remove the last separate operator step.
9. **Recovery.** A member who loses every device has no path back. Recovery witnessed by members is the obvious shape and is not specified.
10. **Collectives.** A community is an admission queue. Giving it an account and motions of its own is designed and not built. In that design the chain itself stays governed by persons, so that opening collectives cannot multiply a vote.
11. **Members of earlier communities,** who joined before this rule existed and were never witnessed in its sense. They are either named as first members of a community or witnessed afresh.
12. **Relations between points.** A point is a digest with no reference to any other. A recorded link from one point to an earlier one, open to challenge like the point itself, is what a share for connecting work and any weight from uptake would both rest on. The opening example is $0.9M$ to the proposer and $0.1M$ divided across the points a matured point links to, the split earlier specifications used. It is not designed.
13. **Starting a network.** The protocol assumes a group that already meets, and asks it for nothing it is not already doing. Outside funding is allowed and sits outside the runtime (Section 14).
14. **Sizing.** Every matured point mints the same amount. Whether members should be able to size a mint to the work, as earlier specifications did, is undecided.
15. **Weight from uptake.** One member, one vote gives a member admitted last week the same say as one whose contributions others have built on for years. A designed extension adds a capped weight from recorded citations, with decay on the citation and not on the person, leaving motions on membership and on the chain's own rules unweighted. A citation would count only when a member records it, where it can be challenged. An attempt to infer uptake from session transcripts failed: talk close to an idea was as common before the idea was first stated as after.
16. **Production constants.** Era, maturity, caps, $V_{\max}$ and M are political as much as technical, and they are for the members to set.
17. **Not yet built:** benchmarked weights, the loader for the real snapshot, and a chain specification with first members' keys held by different people on different machines.

17 Conclusion

One pallet is enough to record who joined, what they put forward and which of it stood, and to mint only when a contribution matures. Keys, members and balances stay three separate maps. Members write without a fee, and nobody else writes except to claim what they already held. Seats go to members in admission order. Opening control ends. The lab runtime does all of this, and its supply reconciles to the unit. Admission mints nothing.

What it does not yet do is bound a colluding set by anything but the cap, limit a member who floods, rotate seats, or let a member authorise an agent to act for them. Nor does it yet let a member vote, challenge or contribute without being named, as Section 12 designs. Those are the next pieces of work.

A Call index

Table 6: Dispatchable calls of `pallet-seeds`. Index 7 is the only unsigned call.

#	Call	Who
0	<code>witness(c, q, e)</code>	member
1	<code>set_keys($keys$)</code>	member
2	<code>propose_point(d, ν)</code>	member, fewer than Q pending
3	<code>challenge(d)</code>	member, not the proposer
4	<code>propose($kind$)</code>	member with no open motion
5	<code>vote(id, aye)</code>	member in the frozen electorate
6	<code>close(id)</code>	member, after the voting period
7	<code>claim($dest, old, \pi$)</code>	unsigned, with an old-key proof

References

- [1] Birdbrain. Seeds: A Store of Values. <https://github.com/Birdbrain-wtf/publications/tree/main/content/articles/seeds-a-store-of-values>, 2026. White paper, companion to this specification.
- [2] Parity Technologies. Polkadot SDK and FRAME. <https://github.com/paritytech/polkadot-sdk>, 2024.
- [3] Parity Technologies. Aura: Authority round consensus. <https://docs.polkadot.com/learn/learn-consensus/>, 2019.
- [4] Alistair Stewart and Eleftherios Kokoris-Kogias. GRANDPA: A Byzantine finality gadget. 2020.
- [5] Birdbrain. Seeds: pallet, runtime, node and end-to-end test. <https://github.com/Birdbrain-wtf/seeds>, 2026.
- [6] Worldcoin Foundation. Worldcoin: a privacy-preserving proof-of-personhood protocol. <https://whitepaper.worldcoin.org/>, 2023.
- [7] Project Individuality. Proof of ink and the polkadot people initiative, 2026. Polkadot forum refs 1783, 1710, 1827.
- [8] Locus. Locus: typed ownership for assets on JAM. <https://forum.polkadot.network/t/18868>, 2025.

- [9] World Wide Web Consortium. Web authentication: An API for accessing public key credentials, level 3. <https://www.w3.org/TR/webauthn-3/>, 2025.
- [10] Privacy and Scaling Explorations. Semaphore: anonymous signalling on ethereum. <https://docs.semaphore.pse.dev/>, 2024.
- [11] Privacy and Scaling Explorations. Rate-limiting nullifier. <https://rate-limiting-nullifier.github.io/rln-docs/>, 2023.
- [12] ZK Email. polkavm-noir-verifier: native polkavm contracts that verify noir proofs. <https://github.com/zkemail/polkavm-noir-verifier>, 2026.
- [13] Birdbrain. Jambo: a reduced JAM client. <https://github.com/Birdbrain-wtf/jambo>, 2026.
- [14] Kusama OpenGov. Kusama JAM upgrade: Option A, lightweight and independent. <https://kusama.polkasassembly.io/referenda/573>, 2025. Wish For Change referendum 573, executed.
- [15] Satoshi Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System. <https://bitcoin.org/bitcoin.pdf>, 2008. White paper.
- [16] Decent Partners. Is There Another Way? <https://zine.decent.partners/articles/is-there-another-way>, 2026. Decent Magazine essay. The point that removing a privileged centre does not remove every centre is Diego Correa Tristain's.
- [17] BrightID. BrightID: a social identity network. <https://brightid.org/>, 2020.
- [18] Proof of Humanity. Proof of humanity: a sybil-resistant registry of humans. <https://proofofhumanity.id/>, 2021.
- [19] E. Glen Weyl, Puja Ohlhaver, and Vitalik Buterin. Decentralized society: Finding web3's soul, 2022.
- [20] Bruno Škvorc. Decaying categorial meritocracy. <https://app.t2.world/article/clp48u2of8539181ymcrc1zjioj>, 2023. Also presented at EthCC 6.
- [21] Bulat Nasrulin, Martijn de Vos, Georgy Ishmaev, and Johan Pouwelse. MeritRank: Sybil tolerant reputation for merit-based tokenomics. 2022.
- [22] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank citation ranking: Bringing order to the web. *Stanford InfoLab Technical Report*, 1998.